

Trabajo de Fin de Ciclo

Entrega final de proyecto — Sistema de Gestión de Incidencias con análisis IA

UTN FRBA — Curso de Inteligencia Artificial para Programadores

Grupo: [COMPLETAR nombre/rol de cada integrante]

Repositorio: <https://github.com/patosimple/Incidencias>

Links de acceso directo (válidos al momento de la corrección)

Recurso	URL
Repositorio GitHub	https://github.com/patosimple/Incidencias
Aplicación en producción	https://incidencias.onrender.com
Video demo	[COMPLETAR enlace — max 3 min]
Otros recursos publicados	[COMPLETAR si aplica — p.ej. el manual de uso /admin de la app]

PARTE 1 — El proyecto como aplicación real

1. Presentación del equipo y del proyecto

Integrantes del grupo (nombre, rol en el desarrollo):

[COMPLETAR por integrante]

Nombre del proyecto

Sistema de Gestión de Incidencias de TI (tickets) con análisis IA asistido.

Problema que resuelve

Gestión interna de bugs y requerimientos sobre dos aplicaciones de negocio (Balances y Financiamiento político). Centraliza el reporte, el seguimiento, la toma colaborativa, el cambio de estado y el cierre de incidentes, e incorpora una capa de análisis IA que asiste a desarrolladores y coordinadores a entender el problema reportado.

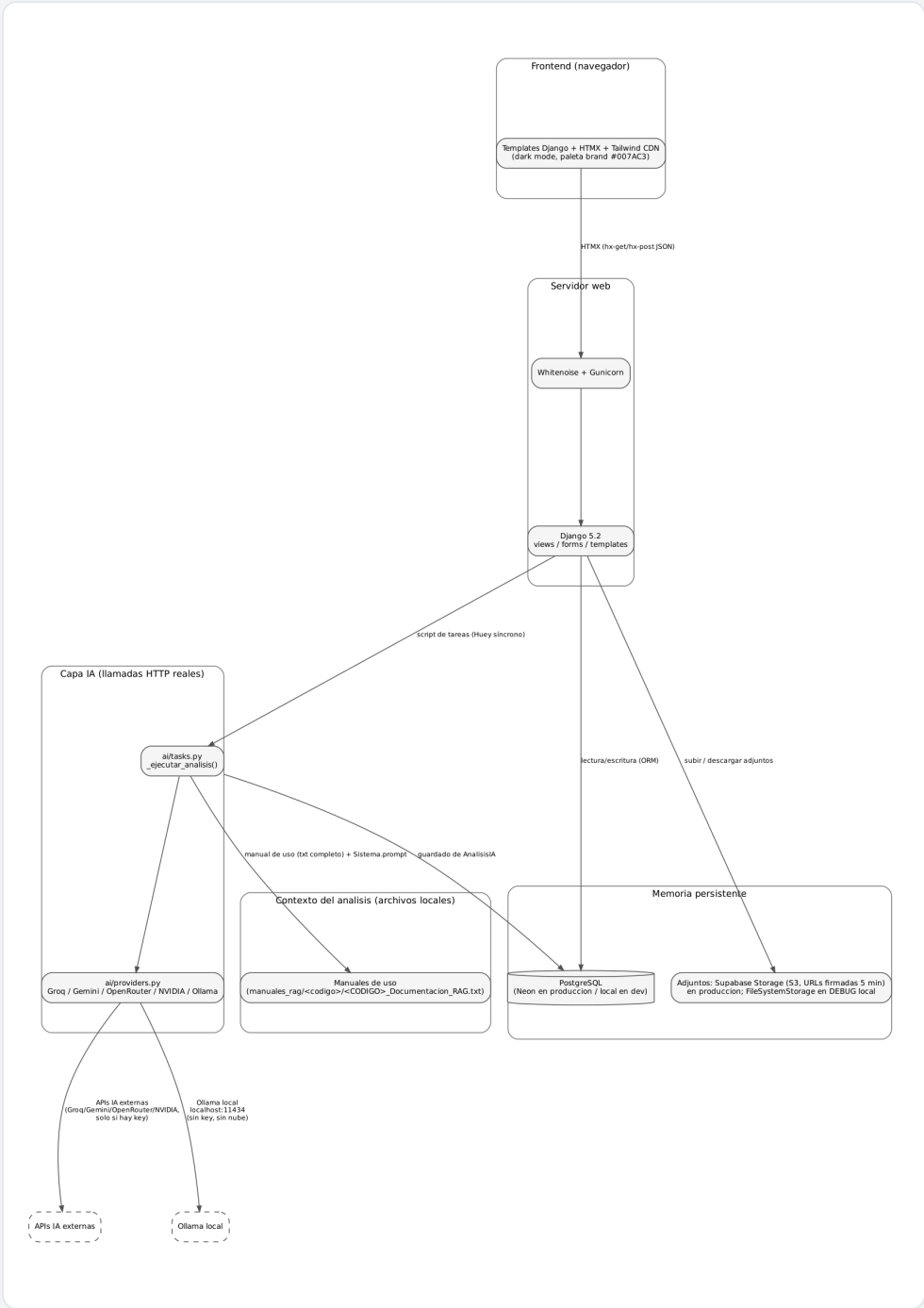
Público objetivo

Usuarios internos de la organización: (a) solicitantes (usuarios de negocio que reportan incidencias sin acceso a detalles técnicos), (b) desarrolladores que atienden los tickets de sus sistemas, y (c) coordinadores que supervisan el flujo. El admin/superuser gestiona usuarios, accesos y configuración de IA.

2. Arquitectura técnica

Diagrama de arquitectura general

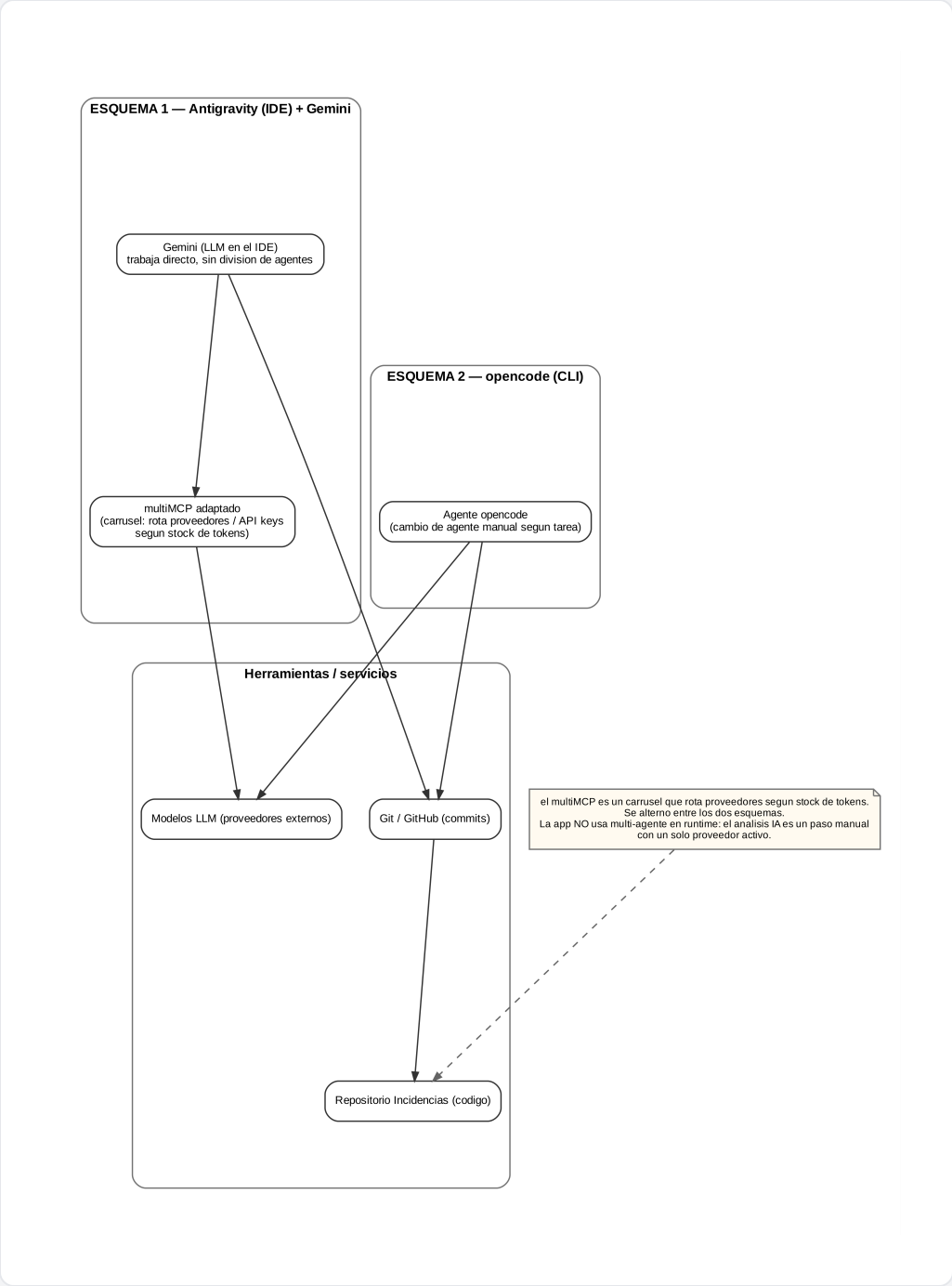
Componentes IA vs lógica tradicional: el núcleo de tickets (crear, listar, comentar, estados, permisos, adjuntos) es lógica tradicional Django. La única componente IA es el módulo de análisis de incidencias (ai/), que se dispara manualmente.



Arquitectura general

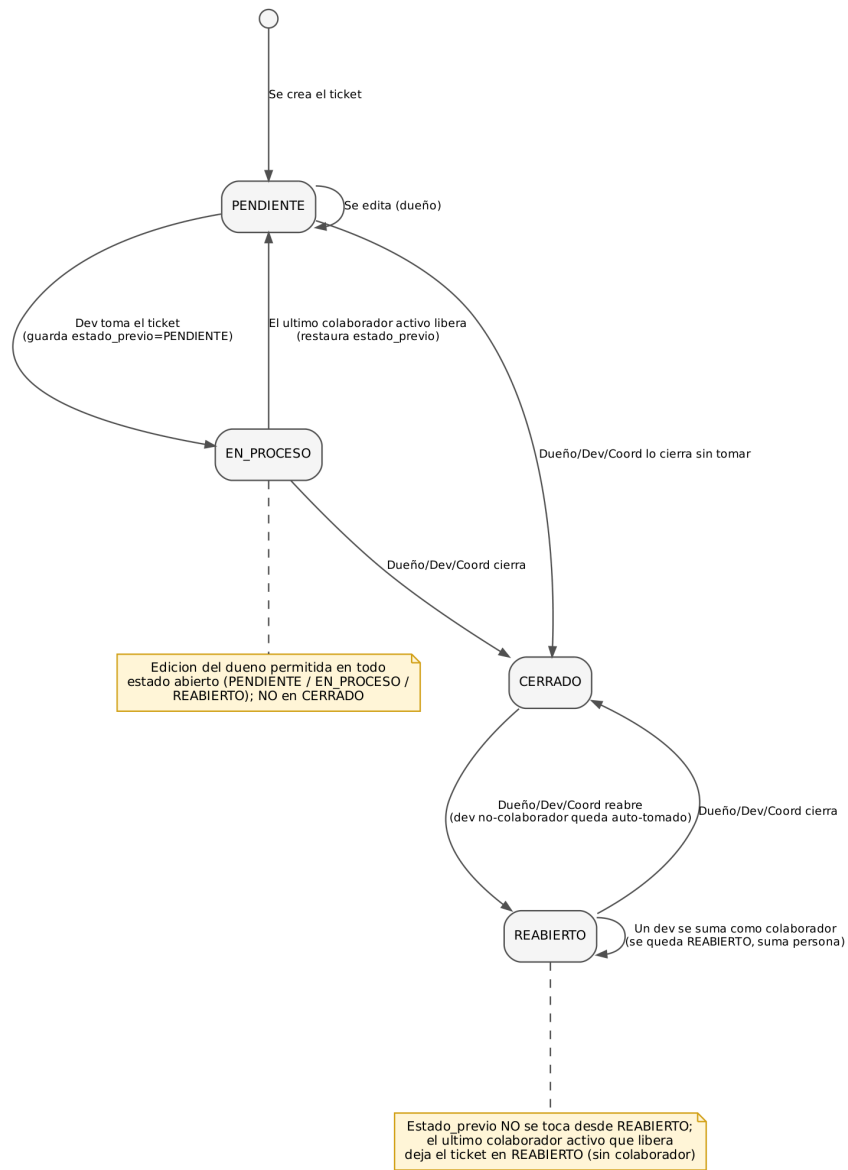
Diagrama de flujo de agentes

El proyecto como aplicación NO incluye orquestación multi-agente en producción: el análisis IA es un paso puntual y manual. El trabajo de desarrollo sí se apoyó en esquemas de IA, alternando dos modalidades: opencode (CLI) con cambio de agente manual por tarea, y Antigravity (IDE) con Gemini como orquestador sobre un multiMCP adaptado para usar rotación interna de API keys y proveedores. El diagrama siguiente representa esta metodología (las partes de la app están cubiertas en los diagramas funcionales).



Flujo de agentes (metodología de desarrollo)

Flujo de estados del ticket



Flujo de estados del ticket

UML — Diagrama de clases

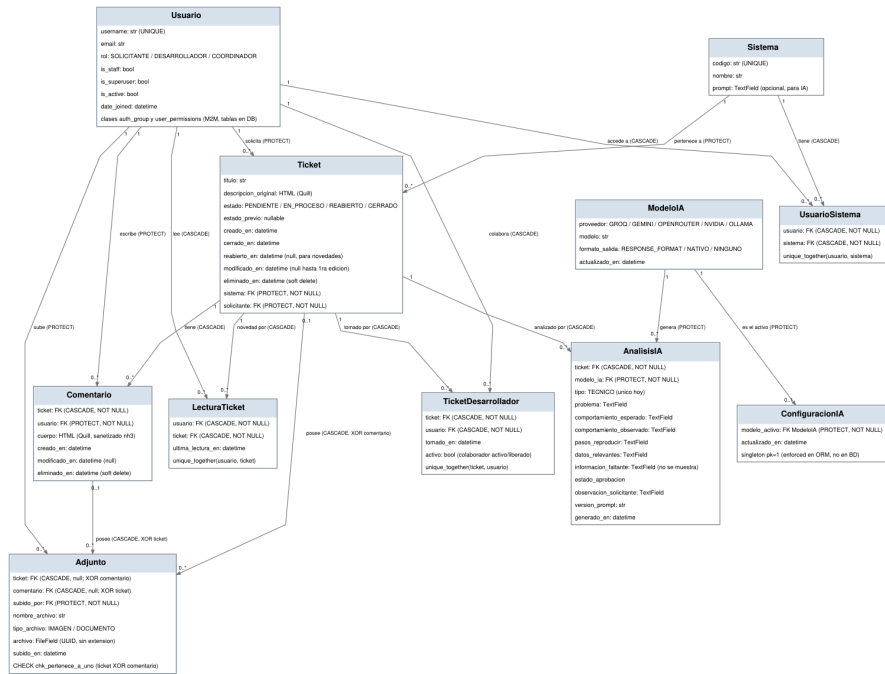
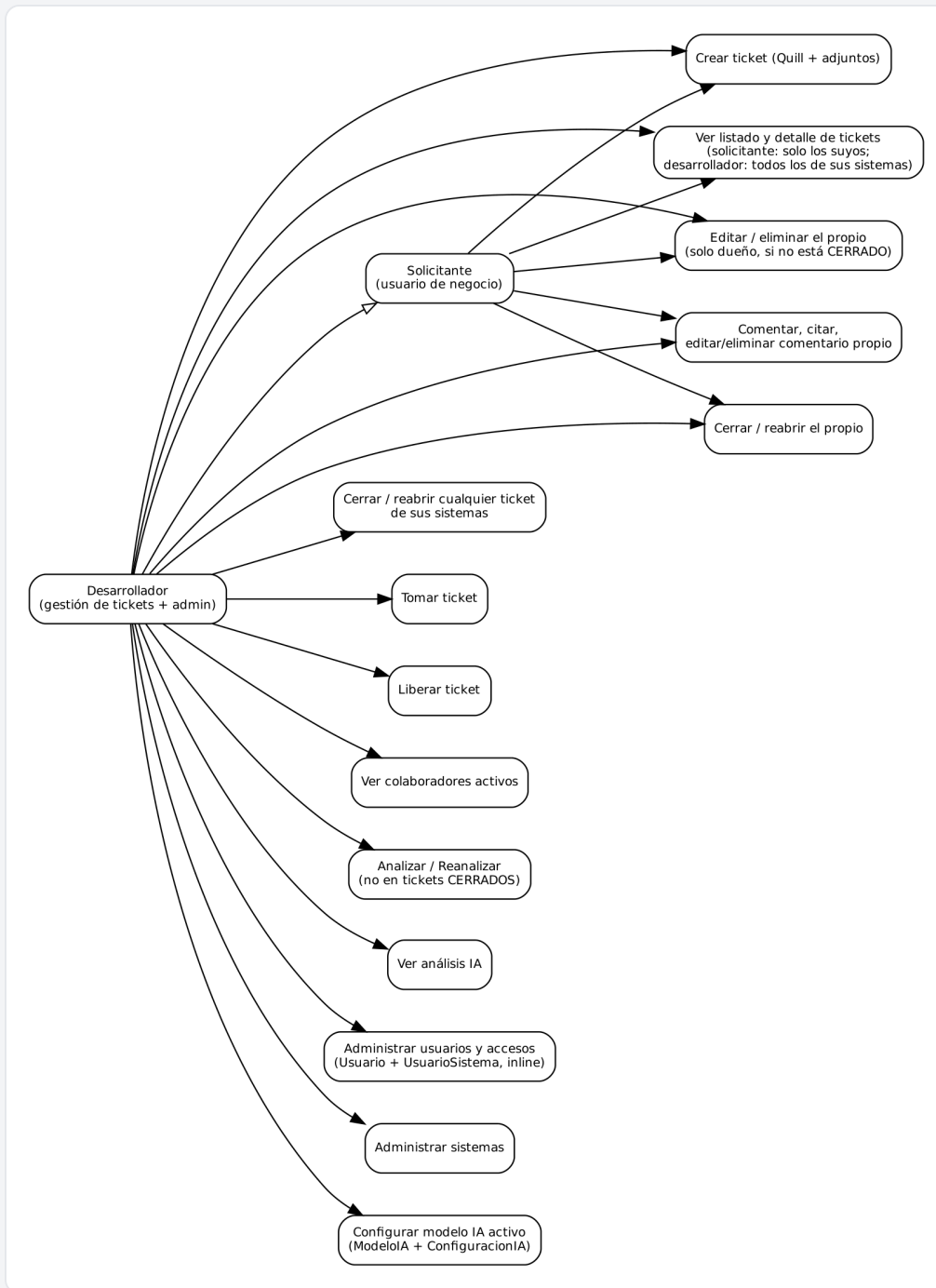


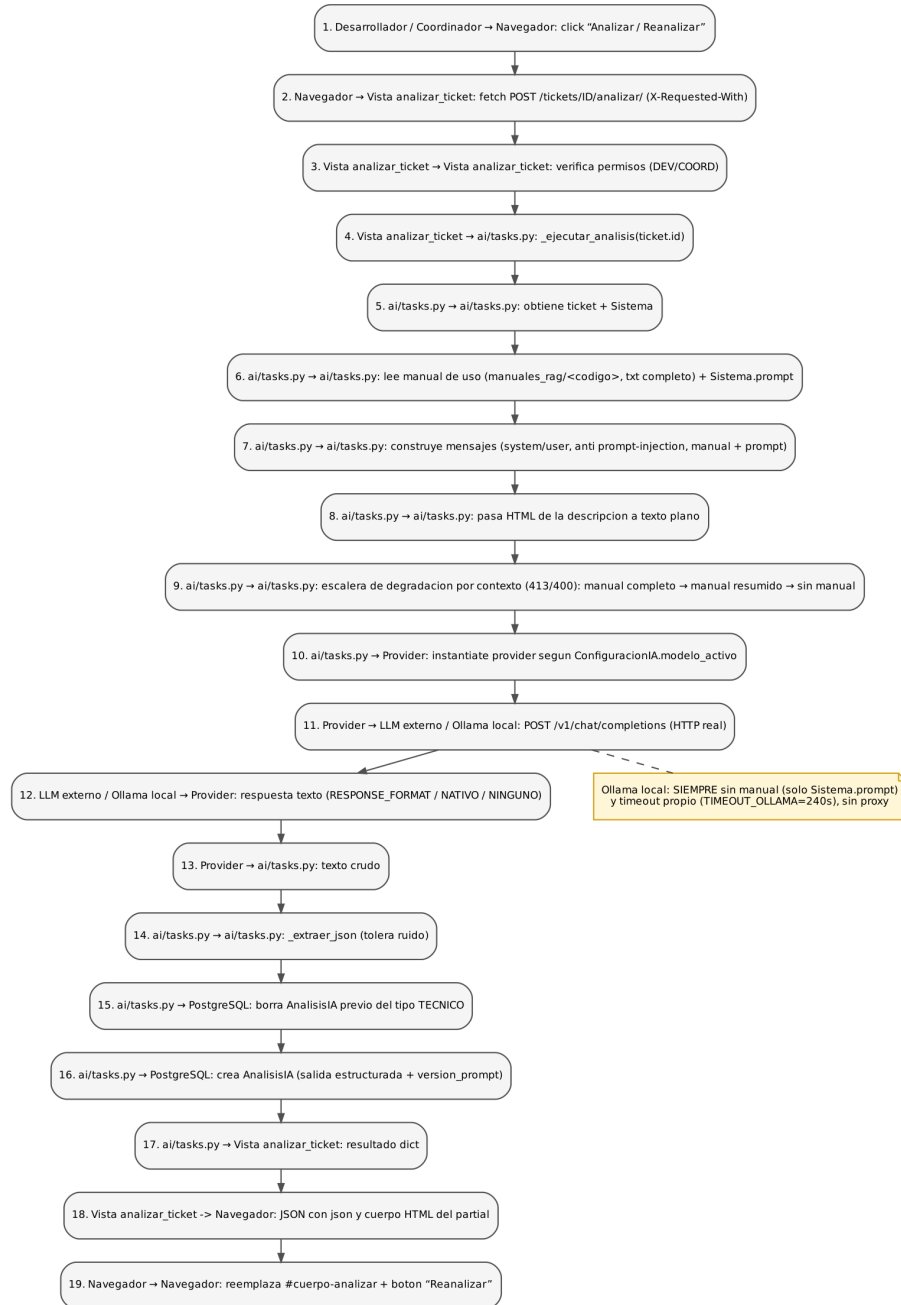
Diagrama de clases (modelo de datos)

UML — Casos de uso



Casos de uso por rol

UML — Secuencia del análisis IA



Secuencia del análisis IA

3. Stack tecnológico

Componente	Tecnología / Herramienta	Por qué esta y no otra
Frontend	Templates Django + HTMX + Tailwind (CDN), dark mode	HTMX permite interacciones parciales sin SPA ni JS pesado; Tailwind da consistencia rápida con paleta de marca y responsive; CDN evita build complejo. Se eligió sobre React/Vue por simplicidad de mantenimiento server-rendered en un equipo chico.
Backend	Python Django 5.2 + Gunicorn + Whitenoise	Django trae ORM, admin, auth y seguridad (CSRF, login) maduros; Python es el stack del curso y el idóneo para integrar APIs de IA. Whitenoise sirve estáticos sin servidor aparte.
Base de datos	PostgreSQL (Neon/Supabase)	Postgres por integridad (constraints, unique_together, índices) y por pgvector (plan RAG futuro). Neon gratis y con SSL. Almacenamiento de adjuntos persistido en Supabase Storage (S3, ver Despliegue). No SQLite porque el deploy en Render usa disco efímero y la app corre persistida en la nube.
Modelo de IA	Multi-proveedor OpenAI-compatible: Groq, Gemini, OpenRouter, NVIDIA; Ollama local. Activo: qwen/qwen3.8-27b (Groq).	Abstracción en ai/providers.py con un único contrato HTTP; permite probar el modelo activo o caer a Ollama local (privacidad). Groq como activo por latencia baja del free tier.
Orquestación	Código propio + Huey (cola de tareas; hoy síncrono con immediate=True)	Sin LangChain: la fachada IA es simple y completa (mensajes, formato, reintentos). Huey+PostgresHuey para no sumar Redis al proyecto: usa la misma base de datos.
Despliegue	Render (Gunicorn) + Neon (Postgres) + Supabase Storage (S3)	Render gratis con deploy por git; Neon gratis sin tarjeta. Los adjuntos van a un bucket S3 privado de Supabase Storage (URLs firmadas 5 min): el disco de Render es efímero y cada redeploy perdería los archivos, por eso se resolvió con storage en la nube (decisión 16/09/2026). Local (DEBUG) usa FileSystemStorage en media/.

4. Evidencia de funcionamiento

Capturas de pantalla (mínimo 3)

Las siguientes capturas son pantallas reales de la aplicación desplegada (login con un usuario del seed). Cubren el mínimo exigido por la consigna: pantalla principal/home, el flujo de uso principal y el resultado del output de IA visible para el usuario.

(a) Pantalla principal / home — listado de tickets con filtros, paginador e indicador de novedades:

Mis Tickets

+ Crear Ticket

REPORTE DE INCIDENCIAS

Mis Tickets

Buscar por título...

Sistema

Estado

Colaborador

#	TITULO	SISTEMA	ESTADO	SOLICITANTE	FECHA
#1	Leyenda Rectificado	Financiamiento	Pendiente	Carlos González	15/09/2026
#2	Lógica en la numeración de notas automáticas	Balances	En proceso	Lucía Fernández	15/09/2026
#3	Error en variación del Estado de Flujo de Efectivo (cambia el signo)	Balances	Cerrado	Valentina Martínez	15/09/2026
#4	Inconsist. en la oferta p/"crear informe" según características del partido.	Financiamiento	Reabierto	María López	15/09/2026
#5	El cuadro del Anexo de Transferencias trascienden los límites de la página del PDF	Balances	En proceso	Joaquín Rodríguez	15/09/2026

Mostrando 1-5 de 5 tickets

← Anterior

Página 1 de 1

Siguiente →

Listado de tickets (home) — rol Desarrollador, tema claro.

(b) Flujo de uso principal — creación de ticket con editor enriquecido y adjuntos:

 FINANCIAMIENTO
AGRUPACIONES POLITICAS

Mis Tickets

+ Crear Ticket

REPORTE DE INCIDENCIAS

 DE 

Nuevo Ticket

[← Volver](#)

Sistema

Titulo

Descripción original

B **I** **U**    

Escriba la descripción del ticket...

Texto enriquecido (HTML/Markdown)

Adjuntos

[+ Adjuntar archivos](#)

Opcional. Podés elegir varios archivos de a uno.

Cancelar

Crear ticket

Creación de ticket (form + editor Quill + lista de adjuntos).

(c) Resultado / output de la IA visible para el usuario — sección Análisis IA en el detalle:

Mis Tickets

+ Crear Ticket

REPORTE DE INCIDENCIAS

Se observaron casos en que si un partido de orden nacional ingresa a la pantalla de creación de informes, no le aparece la opción de crear informe de cargos nacionales (presidente y vice / mercosur nacionales). Este error sucede cuando un partido de orden nacional, no pertenece a una alianza nacional, es decir, en partidos nacionales que puedan participar de manera independiente en la elección.

Colaboradores

Desarrollador

15/09/2026 11:39

Análisis IA

Reanalizar

RESUMEN TÉCNICO

Inconsistencias en la lista de informes disponibles en la pantalla "Crear Informe" según el tipo y dominio del partido (general vs nacional) y su pertenencia a una alianza.

COMPORTAMIENTO ESPERADO

Solo se deben mostrar informes que correspondan al dominio del partido y a los cargos que el partido puede presentar; los partidos nacionales que no pertenecen a una alianza deben poder crear informes de cargos nacionales (presidente, vicepresidente, mercosur) cuando sea pertinente.

COMPORTAMIENTO OBSERVADO

En la pantalla de creación se ofrecen informes que no corresponden al dominio del partido (por ejemplo, informes de otro orden territorial) y, en el caso de partidos nacionales sin alianza, no aparece la opción de crear informes para cargos nacionales.

PASOS PARA REPRODUCIR

1. Ingresar a la aplicación y acceder a la pantalla "Crear Informe". 2. Seleccionar un partido de tipo general y observar los informes listados. 3. Seleccionar un partido nacional sin alianza y observar la ausencia de opciones de cargos nacionales.

DATOS RELEVANTES

Tipo de partido (general o nacional), pertenencia a alianza (sí/no), orden territorial del informe ofrecido, pantalla de creación de informes.

Generado con NVIDIA / openai/gpt-oss-20b - 15/09/2026 12:11

Detalle del ticket con la sección Análisis IA expandida (cards de resultado generado por el LLM).

Video de demostración (opcional)

[COMPLETAR enlace si aplica — max 3 min]

Log de una sesión real

Se incluye en anexo: línea de tiempo de una ejecución completa extraída de la base de datos real (tickets + comentarios + análisis IA con proveedor/modelo/versión de prompt).

```
=====
LOG DE SESION REAL – Sistema de Gestion de Incidencias de TI
Exportado el 16/09/2026 13:29 desde la base de datos de la aplicacion.
Linea de tiempo cronologica: sesion de prueba con datos reales (desde las 13:00 hs).
Incluye: creacion de ticket, comentarios con adjuntos, toma/liberacion, analisis IA, cierre.
=====
16/09/2026 13:11 [11] CREADO ticket #11 "ERROR DE SUMA ALGEBRAICA POR CIFRA NEGATIVA" por María López
(sistema Balances). Estado: Cerrado. Descripcion: Esta incidencia es complicada, principalmente porque no
tenemos una sospecha certera del problema. Involucra varios rubros que no suman bien en pantalla después de
determinada operación con los saldos, pero difiere en los reportes.La principal sospecha, aunque no es
concluyente, podría vincularse con cargar un importe con una cifra negativa. Esto no es la forma habitual de
operar, ya que si quiero bajar un saldo, lo haría por la columna contraria (una cuenta del haber si se trata
de un saldo recomendado del debe y viceversa).Creemos que los usuarios ingresan, en alguna celda o detalle,
una suma con un signo negativo adelante y por eso se rompe la suma lógica, mostrándose mal los totales en las
cuentas madres respecto de las cuentas hijas.Otra posibilidad es el forzado de estos saldos negativos a través
de un modal de carga (AREA, por ejemplo, asume un signo negativo si es un ajuste por gastos, por lo que su
signo negativo está implícito y es inevitable)
16/09/2026 13:11 [11] ADJUNTO a ticket #11: María López subio "CIFRA_NEGATIVA_1.jpg".
16/09/2026 13:11 [11] TOMO ticket #11 Desarrollador Demo (colaborador activo).
16/09/2026 13:11 [11] COMENTARIO Desarrollador Demo en ticket #11: "hola, estoy viendo tu caso, me subis
alguna captura mas?"
16/09/2026 13:12 [11] EDITADO ticket #11 (titulo/descripcion) por María López.
16/09/2026 13:12 [11] ADJUNTO a ticket #11: María López subio "CIFRA_NEGATIVA_2.jpg".
16/09/2026 13:12 [11] COMENTARIO María López en ticket #11: "Respondiendo a Desarrollador Demo:hola, estoy
viendo tu caso, me subis alguna captura mas?listo te agregue la captura que necesitas"
16/09/2026 13:14 [11] ANALISIS IA ticket #11 (tipo TECNICO): modelo Groq/qwen/qwen3.8-27b, version_prompt v6.
Problema: Discrepancia en el cálculo de sumas algebraicas en la vista de saldos cuando se ingresan importes con
signo negativo, afectando la consistencia entre cuentas madre e hijas.
16/09/2026 13:14 [11] COMENTARIO Desarrollador Demo en ticket #11: "perfecto, ya esta solucionado, cierro el
ticket"
16/09/2026 13:15 [11] COMENTARIO María López en ticket #11: "vuelvo a reabrir porque quedó funcionando mal,
lo podes revisar?"
16/09/2026 13:16 [11] COMENTARIO Desarrollador Demo en ticket #11: "Respondiendo a María López:vuelvo a
reabrir porque quedó funcionando mal, lo podes revisar?te pido mil disculpas! revisado y corregido nuevamente.
si te parece que esta ok, podes cerrarlo."
16/09/2026 13:17 [11] COMENTARIO María López en ticket #11: "Respondiendo a Desarrollador Demo:te pido mil
disculpas! revisado y corregido nuevamente. si te parece que esta ok, podes cerrarlo.ahora si! muchas gracias!
cierro el ticket."
16/09/2026 13:17 [11] CERRADO ticket #11 (estado actual Cerrado).
```

5. Evaluación UX/UI

5.1 Heurísticas de Nielsen aplicadas al proyecto

Heurística	¿Cumple?	Evidencia / Observación
Visibilidad del estado del sistema	Sí	Badges de estado coloreados en listado y detalle (PENDIENTE rojo, EN_PROCESO verde, CERRADO gris, REABIERTO); el color distingue si lo tomó el usuario u otro (verde=yo, azul=otro, rojo=sin tomar). Toasts/flash con feedback; spinner + barra de progreso real en adjuntos.
Coincidencia con el mundo real	Sí	Lenguaje del dominio en español (tickets, sistema, solicitante, colaborador); el usuario no ve internals; etiquetas claras en forms y botones con verbos (Tomar, Liberar, Cerrar, Reabrir, Analizar).
Control y libertad del usuario	Sí	Cancelar en todos los forms; quitar adjuntos antes de subir; editar/eliminar comentarios y ticket propio; liberar/reabrir para corregir errores; los enlaces vuelven al listado; limpiar filtros.
Consistencia y estándares	Sí	Paleta brand única (#007AC3) en toda la app, dark mode coherente, partials reutilizables (badges, cards, página de ticket), mismos patrones de botones por estado, misma metodología de filtros en listado.
Prevención de errores	Sí	Validación client-side al seleccionar adjuntos (10MB + whitelist de extensiones con toast), validación de form antes de enviar, botones destructivos con confirmación (SweetAlert2), bloquear submit durante upload, botones contextuales (no aparece 'Tomar' si ya colabora).
Reconocimiento sobre recuerdo	Sí	Selects con label placeholder (Sistema/Estado/Colaborador/Novedades), puntito de novedad por fila, filtro 'Tomado' para devs, badges que se explican solos; el usuario no necesita recordar valores.
Ayuda y documentación	Sí	Manual de uso dentro de la app (/manual/) con 18 secciones colapsables, TOC de anclas y visibilidad por rol (solicitante ve solo lo general; dev/coord suma toma/IA; staff suma admin); FAQs incluidas. Mensajes de error claros en español.

5.2 Evaluación orientada al público objetivo

Diseño apropiado para el nivel técnico: el solicitante no ve internals (ni análisis IA ni toma colaborativa); devs/coord ven las herramientas de gestión sin fricción.

Lenguaje visual/textual comprensible: español de negocio, no jerga de backend; hint de adjuntos y textos de acción verbales. El análisis IA pide solo información que el usuario podría aportar (pantalla, pasos, mensaje de error, frecuencia, OS).

Prueba con usuario real: se corrieron escenarios reales con el usuario y de ahí salieron decisiones de UX concretas: toasts para mutaciones in-place (comentarios) vs flash tag cuando hay navegación; límite de adjuntos en 10 MB (los informes llegan hasta ~6 MB); indicador de novedades como puntito azul (en vez de badge) con toggle en el header; no mostrar 'información faltante' del análisis IA (sin valor para el lector técnico); filtros con label placeholder en español.

6. Evaluación de Ciberseguridad

Riesgo identificado	Tipo (OWASP/privacidad/acceso)	Medida implementada
Inyección de prompt en el modelo IA	Prompt injection	Prompt separado system/user; el user declara que la descripción del ticket es SOLO dato (puede contener órdenes) que se ignoran; bloque delimitado con "" y defensa explícita en system; el título también se protege. Ninguna instrucción dentro de la descripción modifica la tarea/reglas/formato.
Exposición de API keys	Secretos en código	Keys en .env (ignorado en git) propagadas a os.environ en settings; nunca hardcodeadas; .env.example commiteado solo con placeholders.
Datos de usuarios almacenados	Privacidad	Modelo mínimo: username, rol, accesos a sistemas; no se guardan datos sensibles extra; contraseñas hasheadas por Django; el análisis IA con datos de Financiamiento político puede correr 100% local con Ollama para no salir de la org.
Acceso no autorizado (autenticación/autorización)	Acceso	Login obligatorio (LOGIN_URL), permisos por rol (solicitante ve solo sus tickets; dev ve sus sistemas), verificación server-side (403/404) en tomar/cerrar/reabrir/comentar/analizar; superuser con bypass controlado; soft-delete conserva auditoría. Se implementó integralmente.
XSS en contenido del ticket/comentarios	OWASP A3	Sanitización nh3 al guardar (whitelist de tags/atributos Quill) en crear/editar ticket y comentarios; render con safe es seguro porque el input llega ya saneado; tests E2E de XSS automatizados (XssVistaTest).
Adjuntos maliciosos	OWASP A1 (mayormente)	Whitelist de extensiones (imágenes, office, comprimidos) — excluidos ejecutables, .svg, macros de Office y

Riesgo identificado	Tipo (OWASP/privacidad/acceso)	Medida implementada
		html/xml; límite de 10MB por archivo con validación client y server.
CSRF / sesión	OWASP A1/A8	CSRF middleware activo y token en todos los forms POST (incluidos los HTMX); logout por POST.

7. IAs usadas en el co-work de desarrollo

El desarrollo se hizo en dos esquemas combinados: (1) esquema multi-agente con Antigravity + Gemini como orquestador + un multiMCP adaptado para usar rotación interna de API keys y proveedores; y (2) opencode como CLI con cambio de agente manual por tarea. Ambas modalidades se alternaron a lo largo del desarrollo y se complementaron (no hay un reparto de features por esquema: el mismo hito se pudo abordar desde cualquiera de las dos). El contexto original del proyecto quedó documentado en el archivo AGENTS.md dentro del repositorio, que ambas herramientas leyeron y actualizaron; así el paso de un esquema al otro no perdía la memoria del desarrollo (conceptos, decisiones y pendientes).

Herramienta IA	Para qué la usaron	Aportó
Claude (claude.ai, chat)	Planificación y diseño del sistema (entidades, flujos, esquema de datos) directamente desde el chat de claude.ai, y allí mismo generó el proyecto Django inicial listo para trabajar: estructura del proyecto, modelos, urls y views iniciales, archivos docker y requirements.txt, entregados como .zip.	Bien: el andamiaje completo del proyecto quedó definido y descargable desde el arranque y consistente con el diseño planificado; sobre esa base luego se trabajó con AntigraVity (multi-agente) y opencode (agentes manuales).
AntigraVity + Gemini (orquestador multi-agente) + multiMCP adaptado con rotación interna de API keys y proveedores	Desarrollo de parte del código con esquema multi-agente: varios agentes orquestados por Gemini para generar features del backend y frontend.	Bien: permitió encarar el desarrollo con agentes especializados coordinados por un orquestador. Limitaciones del multiMCP: en la sesión de prueba varios proveedores fallaron (modelos deprecados, carrusel de proveedores sin saldo, 404/410 en NVIDIA), por lo que el trabajo se terminó combinando con opencode manual.
opencode (CLI, agentes manuales)	Generar backend Django, forms, vistas, templates (Tailwind/HTMX/Quill), capa IA multi-proveedor, tests automatizados, debugging	Bien: velocidad de generación de estructura completa y consistente; revisión de bugs con repro (p.ej. scroll de SweetAlert2, orden de serialización HTMX). Mal: a veces generaba código con errores sutiles (p.ej. {# #} multilínea renderizado literal, orden de listeners HTMX) que hubo que corregir con tests.
GitHub/Git (historial de commits)	Evidencia del proceso real de trabajo incremental	Bien: historia de commits real es requisito de evaluación.

Reflexión obligatoria

Sin el co-work con IA el desarrollo hubiera tomado al menos el doble de tiempo: la capa IA multi-proveedor (providers OpenAI-compatible con formato de salida por modelo, prompt anti prompt-injection, escalera de degradación por contexto), la suite de ~65 tests automatizados (XSS, adjuntos, transiciones, novedades, permisos; más de 120 al cierre) y los fixes finos de UX (scroll del SweetAlert2, orden de serialización HTMX, barra

de progreso real en adjuntos). La IA también falló y esas fallas se corrigieron: un comentario multilínea `{# #}` de Django que se renderizaba literal, incidentes de orden de listeners HTMX que a veces no actualizaban el detalle, y garantías de XSS inexistentes que los propios tests delataron (el `render |safe` solo es seguro si el input se sanea al entrar). Sobre los esquemas: el multi-agente (Antigravity/Gemini) ayudó a encarar el código con agentes especializados coordinados, pero su dependencia del carrusel de proveedores resultó frágil en la sesión de prueba; el cambio manual de agente en opencode dio más control y trazabilidad del trabajo por tarea, y fue el esquema con el que se terminaron de consolidar las features y los tests.

PARTE 2 — IA local en tu proyecto

Nota: la aplicación ya incorpora Ollama como proveedor local (formato NATIVO, sin key). Como contexto del análisis, el prompt se arma con la descripción del ticket, la descripción oficial del sistema (`Sistema.prompt`) y el manual de uso del sistema en texto plano (`manuales_rag/<codigo>`, leído al momento de analizar). Se evaluaron dos variantes para incorporar los manuales: RAG vectorial (embeddings locales con `nomic-embed-text + pgvector`) y enviar el manual completo en el prompt. Se eligió esta última porque los manuales son acotados (`BALANCES ~24 KB`, `FINANCIAMIENTO ~16 KB`, `~10-12K tokens`) y entran enteros en el contexto: el modelo ve todo el contenido, sin riesgo de fragmentos mal seleccionados por similitud; además evita `pgvector`, el pipeline de indexado y la carga de un modelo de embeddings en Ollama por análisis. El RAG quedó documentado como plan para escalar a manuales grandes.

Pregunta 1 — Qué papel jugaría un LLM/SLM local

Reemplazaría (o complementaría) la API externa para escenarios sensibles: los tickets de Financiamiento político no deberían salir de la organización. El provider Ollama ya existe en la app: cambiando el modelo activo (`ConfiguracionIA`) el análisis corre local. Hoy el contexto del análisis ya incluye el manual de uso del sistema (txt completo, sin que los datos salgan de la org); a futuro podría sumarle embeddings locales (`nomic-embed-text`) para RAG sobre esos manuales. Sería un componente de soporte intercambiable, no el agente principal.

Pregunta 2 — Qué le aportaría al usuario

Privacidad (los datos sensibles no salen de la org), cero costo por token y funcionamiento offline. En experiencia, permitiría ofrecer el análisis como garantía por defecto para incidencias de sistemas sensibles, sin depender del estado de un proveedor externo.

Pregunta 3 — Qué te aportaría a vos como profesional

Analizar logs/comportamientos de incidencias y patrones de reportes sin necesidad de que los datos salgan de la organización; probar el pipeline de IA offline; el modelo local corre con el mismo código (mismo prompt y formato de salida), lo que permite desarrollo sin keys ni costo y validar privacidad ante el cliente.

Pregunta 4 — Limitaciones concretas vs API en la nube

Comparación de tiempos de análisis local vs nube (medido sobre un ticket real con el prompt completo y el manual del sistema; [COMPLETAR con las mediciones reales]): el análisis con Ollama local (qwen2.5:3b-instruct-q4_K_M, ~6 tok/s de decode) tarda [T_LOCAL — ~90-120s] vs la API en la nube (Groq, qwen/qwen3.8-27b) que tarda [T_NUBE — pocos segundos]. La implementación final está prevista sobre un servidor de la organización con características superiores a los equipos de desarrollo, lo que reduce la brecha de rendimiento. La calidad del modelo local de ~3B es inferior a la de los ~27B de Groq para el caso de uso específico. Mantenimiento (actualizar el modelo, ollama pull) recae en el equipo local. No conviene para volumen alto ni para tareas globales largas.

Entregable opcional — captura de Ollama local

[COMPLETAR captura tras ejecutar, p.ej.:]

Comando: **ollama run qwen2.5:3b**

Pregunta: "En una organización que gestiona incidencias de Financiamiento político, qué ventajas y riesgos tiene analizar esas incidencias con un LLM local versus una API en la nube?"

Respuesta (1 línea): [COMPLETAR lo que respondió el modelo]